

ACM India Summer School on Symmetric Key Cryptography

Day 8: Cryptanalysis of FEAT PACQARMA3 and Machine Learning-based Cryptanalysis of Speck32/64

Sourav Sharma

June 15, 2026

1 Introduction

Day 8 of the ACM India Summer School on Symmetric Key Cryptography focused on two advanced areas of symmetric key cryptanalysis:

1. **Arm’s FEAT PACQARMA3 Feature:** Highlighting key recovery attacks on the QARMA3 tweakable block cipher under practical settings such as the Pointer Authentication Code (PAC) feature in ARM architecture, which introduces data clamping and output chopping constraints.
2. **Machine Learning-based Cryptanalysis of Speck32/64:** Reviewing Aron Gohr’s pioneer CRYPTO 2019 work, which demonstrates how deep residual neural networks (ResNets) can act as differential distinguishers that outperform classical baseline classifiers built using the full Difference Distribution Table (DDT).

To connect theory to experiment, I implemented the Speck32/64 block cipher and simulated the differential propagation of the standard Gohr input difference $\Delta_{\text{in}} = (0x0040, 0x0000)$ across round-reduced ciphers.

2 Arm’s FEAT PACQARMA3 and Key Recovery Attacks

2.1 The PAC Feature and QARMA3 Block Cipher

The Pointer Authentication Code (PAC) is a hardware security mechanism designed to enforce Control Flow Integrity (CFI). It works by computing a cryptographic tag (using a tweakable block cipher) of a pointer address and its context, storing this tag in the unused upper bits of the 64-bit pointer. Upon pointer dereference, the tag is verified. QARMA is a family of lightweight tweakable block ciphers designed specifically for this latency-critical application. QARMA3 refers to the 8-round version of the cipher. In the PAC environment, the cipher operates under two severe constraints:

- **Clamping:** The pointer address (plaintext input) has its unused upper bits (e.g., $64 - \text{VA}$ bits, where VA is the Virtual Address size) clamped to 0. The attacker cannot introduce differences in these clamped bits.
- **Chopping:** The observer can only view the computed tag, which is a truncated version of the ciphertext (typically only the upper $64 - \text{VA}$ bits).

2.2 Differential Attacks under Clamping and Chopping

Despite the constraints of clamping and chopping, recent cryptanalytic techniques have shown that QARMA3 is vulnerable to related-tweak differential attacks. The general attack model utilizes:

1. **Pair Sieving:** Given structures of plaintext-tweak pairs, candidates are filtered based on whether their chopped ciphertext difference matches a target output difference set D_{out} .

2. **Guess-and-Filter:** The attacker guesses parts of the last round key and decrypts the ciphertexts backwards by one round. If the partially decrypted difference match a target middle difference Δ_{in} , the key guess counter is incremented.

The success probability is governed by the signal-to-noise ratio, where the signal corresponds to the actual differential characteristic probability p and the noise corresponds to the random association of chopped ciphertexts. Depending on the size of the Virtual Address space ($\text{VA} = 32$ to 56 bits), the key recovery attack succeeds with a time complexity significantly below the security bounds of QARMA3.

3 Machine Learning-based Cryptanalysis of Speck32/64

3.1 Speck32/64 Block Cipher

Speck32/64 is a lightweight ARX (Addition-Rotate-XOR) block cipher designed by the NSA. It operates on 32-bit blocks (divided into two 16-bit words L_i, R_i) and uses a 64-bit key (four 16-bit words). The round function is defined as:

$$\begin{aligned} L_{i+1} &\leftarrow ((L_i \ggg 7) + R_i) \oplus K_i \pmod{2^{16}} \\ R_{i+1} &\leftarrow (R_i \lll 2) \oplus L_{i+1} \end{aligned}$$

where K_i is the round key. The key schedule uses the same round transformations to update the key words.

3.2 Classical DDT vs. Neural Distinguishers

Classical differential cryptanalysis treats the cipher as a Markov chain where the key is random, analyzing only the XOR difference $\Delta_{\text{out}} = C_0 \oplus C_1$. It assumes that all ciphertext pairs sharing the same difference are equally likely to have originated from the input difference. A classical distinguisher uses the Difference Distribution Table (DDT) and computes:

$$P(\Delta_{\text{out}}|\text{Real}) > P(\Delta_{\text{out}}|\text{Random})$$

In CRYPTO 2019, Aron Gohr trained a deep ResNet (taking the raw ciphertext pair C_0, C_1 as input, represented as four 16-bit words) to distinguish real ciphertext pairs from random ones under a fixed input difference $\Delta_{\text{in}} = (0x0040, 0x0000)$. Gohr showed that the neural network learns to reject **”imposter pairs”**—pairs whose XOR difference is a highly probable output difference, but whose specific values cannot have reasonably propagated from the input difference under any key. In the **”Real Differences Experiment,”** the neural network achieved 70.7% accuracy in distinguishing real pairs from random pairs having the exact same XOR difference distribution, proving that it exploits features beyond the classical DDT.

4 C++ Simulation and Experimental Verification

I implemented Speck32/64 and simulated the propagation of the input difference $\Delta_{\text{in}} = (0x0040, 0x0000)$ in `speck.cpp`. The implementation was first validated using the official NSA test vector:

- **Key:** 0x1918111009080100
- **Plaintext:** 0x6574694c
- **Ciphertext:** 0xa86842f2

The simulation ran 10^7 random plaintext-key pairs for round counts $r = 1, 2, 3, 4, 5$.

4.1 Experimental Results

The simulation computed the empirical differential probabilities for the most frequent output differences:

Rounds	Highest Difference ($\Delta L, \Delta R$)	Emp. Prob.	Theo. Prob.
1	(0x8000, 0x8000)	1.0000	1.0 (2^0)
2	(0x8100, 0x8102)	0.5003	0.5 (2^{-1})
3	(0x8000, 0x840a)	0.1249	0.125 (2^{-3})
4	(0x850a, 0x9520)	0.00785	0.00781 (2^{-7})
5	(0x802a, 0xd4a8)	0.000127	0.000122 (2^{-13})

Table 1: Empirical differential propagation probabilities for Speck32/64 starting from $\Delta_{\text{in}} = (0x0040, 0x0000)$.

4.2 Analysis

- **Round 1:** The input difference $(0x0040, 0x0000)$ rotates to $0x8000$ in the first step. Since it is located at the MSB, addition with R_0 produces no carries, meaning the difference propagates with probability 1.0 to $(0x8000, 0x8000)$.
- **Rounds 2–3:** The difference propagates along the high-probability characteristics with very little noise, yielding probabilities close to 2^{-1} and 2^{-3} respectively.
- **Rounds 4–5:** As the number of rounds increases, carry effects and diffusion cause the differential path probability to drop to 2^{-7} and 2^{-13} . These matches Gohr’s baseline DDT distinguisher capabilities, proving that the simulation correctly models standard Speck32/64 differential propagation.